

练习0：填写已有实验

在本实验中，我们对前几次实验的代码进行了复用和必要的修改。大部分代码基于之前实验的成果，主要修改集中在中断处理相关部分。

主要修改内容

trap.c 文件的时钟中断处理部分

在 trap.c 文件的时钟中断处理部分，我们进行了关键性修改：

```
case IRQ_S_TIMER:
    clock_set_next_event();
    if (++ticks % TICK_NUM == 0 && current) {
        current->need_resched = 1;
    }
    break;
```

修改背景与问题分析

我们在后续 make grade 的时候发现，我们的 spin.c 文件总是无法输出后续的内容，经过我们对于子进程与父进程的调换分析，我们识别到问题的根源：子进程在执行过程中未能正确地将CPU控制权交还给父进程。

这样的问题就会导致当子进程进入 while(1) 无限循环后，由于缺乏有效的中断调度机制，子进程持续占用CPU资源，导致父进程无法获得执行机会，从而无法输出后续的验证信息。

所以进行再几步的分析，我们确定问题的根源在于时钟中断处理函数未能正确触发进程调度机制。原来的中断处理逻辑可能缺少对进程重调度标志的设置，导致调度器无法在适当时机介入并进行必要的进程切换。

解决方案设计

我们设计的解决方案的核心思想是**通过完善时钟中断处理来强制进行周期性的进程调度**。

具体而言，在每次时钟中断发生时，系统首先通过 clock_set_next_event() 函数设置下一次中断时间，维持系统的时间基准。然后，通过全局计数器 ticks 来跟踪时间流逝，当累积的时钟滴答数达到预设的 TICK_NUM 阈值时，系统会主动设置当前进程的 need_resched 标志。

这一设计确保了即使某个进程陷入无限循环，也能在时间片用尽时被强制剥夺CPU使用权。设置 need_resched 标志相当于向调度器发出明确信号，表明当前进程已经使用了足够的时间片，应该让出CPU给其他就绪进程执行。这种基于时间片的抢占式调度机制，有效防止了单个进程垄断CPU资源的情况发生。

同时，这种解决方案还考虑了系统的整体性能平衡。通过合理设置 TICK_NUM 参数，我们可以在调度开销和系统响应性之间找到最佳平衡点。较小的 TICK_NUM 值会提高系统的响应速度，但会增加调度开销；较大的值则相反。经过测试验证，当前设置的参数100能够在保证系统性能的同时，有效解决进程饥饿问题。

练习1: 加载应用程序并执行（需要编码）

一、代码实现

第一部分：在 `alloc_proc` 中初始化新增的 `proc_struct` 成员

在 `alloc_proc` 函数中，我们对进程控制块的新增字段进行初始化，为进程管理奠定基础：

```
proc->wait_state = 0;           // 等待状态初始化为0
proc->cptr = NULL;              // 子进程指针初始化为NULL
proc->yptr = NULL;              // 弟弟进程指针初始化为NULL
proc->optr = NULL;              // 哥哥进程指针初始化为NULL
```

核心设计思路：

1. `wait_state` 字段初始化

将 `wait_state` 初始化为0，表示新创建的进程初始状态下不处于任何等待状态

这是进程调度和同步机制的基础，确保进程在创建后可以立即进入就绪队列等待调度，并且后续当进程需要等待某些事件（如子进程退出、资源可用等）时，该字段会被设置为相应的等待状态。

2. 进程关系指针初始化

- `cptr`（子进程指针）：初始化为NULL，表示新进程暂时没有子进程，为后续通过 `fork` 系统调用创建子进程做好准备。
- `optr`（哥哥进程指针）：初始化为NULL，表示在兄弟进程链表中还没有年长的兄弟进程。
- `yptr`（弟弟进程指针）：初始化为NULL，表示在兄弟进程链表中还没有年幼的弟弟进程。

我们新添加的这些指针共同构成了 `ucore` 的进程家族树结构，支持父子进程关系和兄弟进程关系的维护。

第二部分：在 `load_icode` 中设置 `trapframe` 内容

在 `load_icode` 函数的第6步中，我们设置 `trapframe` 来构建完整的用户态执行环境：

```
// 设置用户栈指针 - 指向用户栈顶部
tf->gpr.sp = USTACKTOP;

// 设置程序计数器 - 指向ELF入口点
tf->epc = elf->e_entry;

// 设置状态寄存器 - 为用户模式配置
// SPP: 前一个特权级模式 (0=User, 1=Supervisor)
// SPIE: 前一个中断使能状态
// 我们需要设置状态为用户模式，并启用中断
tf->status = (sstatus & ~SSTATUS_SPP) | // 清除SPP位，设置为用户模式
             SSTATUS_SPIE;             // 设置SPIE，在用户模式下启用中断
```

核心设计思路：

1.用户栈空间设置

在用户执行环境的构建过程中，用户栈空间的设置是确保程序正确运行的基础保障。

```
tf->gpr.sp = USTACKTOP;
```

通过将trapframe中的栈指针设置为** USTACKTOP，我们为用户程序建立了一个完整且安全的栈内存区域**。

这一设计的核心考量在于，任何用户程序的执行都离不开函数调用链的支持，而函数调用必然需要栈空间来保存返回地址、局部变量和函数参数等关键信息。所以我们将栈指针初始化为用户地址空间的顶部地址，采用**向下生长的栈设计模式**，能够充分利用预先分配的用户栈空间（从 USTACKTOP - USTACKSIZE 到 USTACKTOP），同时通过内存保护机制**防止栈溢出对系统其他部分造成影响**。

这种设置不仅为普通的函数调用提供了支持，也为可能的异常处理、信号处理等高级特性预留了必要的栈框架空间。

2.程序入口点设置

程序入口点的正确设置是实现用户程序从预定位置开始执行的关键环节。

```
tf->epc = elf->e_entry;
```

通过将trapframe中的epc寄存器设置为ELF文件头中指定的入口地址 elf->e_entry，我们建立了从内核加载器到用户应用程序的无缝连接。

这个入口地址指向用户程序的main函数或其他指定的起始执行点。

这种设计确保了当CPU从内核态切换到用户态时，指令执行流程能够准确地跳转到应用程序的逻辑起点，从而开始正常的程序执行。除此以外，这种机制保证了所有用户程序都遵循统一的启动规范，为操作系统的进程管理提供了一致性和可预测性。

3.处理器状态设置

处理器状态的配置是实现安全特权级切换的核心技术。

```
tf->status = (sstatus & ~SSTATUS_SPP) | SSTATUS_SPIE;
```

通过将trapframe中的status寄存器设置为 (sstatus & ~SSTATUS_SPP) | SSTATUS_SPIE，我们实现了两个重要的安全目标：

(1) 清除SPP位，将前一个特权级设置为用户模式，这确保了在执行eret指令后CPU能够从内核特权级安全地降级到用户特权级，建立了必要的安全边界；

(2) 设置SPIE位，保证了在用户模式下中断处于使能状态，这使得用户程序能够正常响应系统调用、时钟中断等外部事件，同时又不破坏系统的整体稳定性。

这种状态设置体现了在安全性和功能性之间的精细平衡，既保护了内核空间不受用户程序的非法访问，又为用户程序提供了必要的系统服务支持。

4.总体思路

整个trapframe的设置遵循"执行环境预设"的设计理念：

- 代码环境：通过epc指定执行起点
- 数据环境：通过sp提供栈空间
- 控制环境：通过status建立安全执行环境

这三者共同构成了用户进程执行的完整环境，通过 `eret` 指令的原子性操作实现从内核态到用户态的安全平滑过渡。

二、请简要描述这个用户态进程被ucore选择占用CPU执行（RUNNING态）到具体执行应用程序第一条指令的整个经过。

1.进程调度与上下文切换

当ucore的调度器选择该用户进程执行时，系统进入关键的进程切换阶段。调度器调用 `proc_run` 函数，这个函数负责完成从当前运行进程到目标用户进程的完整切换过程。

在切换过程中，系统首先加载目标进程的页表基地址，通过 `lsatp(proc->pgdir)` 指令更新页表寄存器，确保后续内存访问使用正确的地址空间。接着执行 `switch_to` 函数完成上下文切换，这个操作保存当前进程的寄存器状态，同时恢复目标进程之前保存的执行上下文。

上下文切换完成后，CPU的执行流程跳转到新进程的内核线程起点，开始在新的进程上下文中运行。此时虽然已经切换到目标进程，但仍在内核特权级下执行系统管理代码。

2.返回到用户态的准备工作

我们的进程切换后首先执行的是 `forkret` 函数，这是所有新创建进程在内核中的统一入口点。`forkret` 进一步调用 `forkrets(current->tf)`，开始准备从内核态返回到用户态的关键步骤。

`forkrets` 函数接收当前进程的trapframe指针作为参数，这个trapframe包含了之前在 `load_icode` 中精心设置的用户态执行环境。函数将trapframe指针调整到内核栈的适当位置，为后续的寄存器恢复操作做好准备。

随后执行流程跳转到 `__trapret` 汇编代码段，这是实现特权级切换的核心环节。`__trapret` 开始从trapframe中逐个恢复通用寄存器的值，建立完整的用户程序执行环境。

3.关键寄存器恢复与特权级切换

在 `__trapret` 阶段，系统依次恢复所有关键寄存器状态。

首先恢复通用寄存器，确保用户程序有正确的数据环境；接着恢复用户栈指针 `sp`，将其设置为 `USTACKTOP`，为用户程序提供可用的栈空间；然后恢复程序计数器 `epc`，指向ELF文件的入口地址 `elf->e_entry`；最后恢复状态寄存器 `status`，其中包含了特权级和中断使能的配置信息。

当所有寄存器恢复完成后，CPU执行**`eret`指令，这是一个原子性的特权级切换操作。在 `eret` 执行的瞬间，多个硬件级别的变化同时发生：处理器的特权级从内核模式（S-mode）降级到用户模式（U-mode）**，程序执行流程跳转到 `epc` 寄存器指向的应用程序入口地址，内存管理单元开始使用用户页表进行地址转换，中断处理机制切换到用户模式下的配置。

4.用户程序开始执行

完成特权级切换后，CPU开始在用户特权级下执行应用程序的第一条指令。此时处理器完全运行在用户模式，所有内存访问都通过用户页表进行地址转换，有效实现了内核空间与用户空间的隔离保护。

用户程序使用预设的用户栈空间进行函数调用和局部变量存储，栈指针指向 `USTACKTOP` 位置。系统调用通过 `ecall` 指令触发，陷入内核态由操作系统提供服务。外部中断和异常按照用户模式的配置进行处理，确保系统的响应性和稳定性。

整个过程中，通过精心设计的 `trapframe` 结构和硬件支持的特权级切换机制，实现了从内核管理到用户执行的平滑过渡，为用户程序提供了一个安全、隔离且功能完整的运行环境。

练习2: 父进程复制自己的内存空间给子进程 (需要编码)

一、补充 `copy_range` 的实现

`copy_range` 函数负责将父进程的用户内存空间内容复制到子进程中，是 `do_fork` 执行过程中的关键步骤。

```
int copy_range(pde_t *to, pde_t *from, uintptr_t start, uintptr_t end,
               bool share)
{
    assert(start % PGSIZE == 0 && end % PGSIZE == 0);
    assert(USER_ACCESS(start, end));
    do
    {
        pte_t *ptep = get_pte(from, start, 0), *nptep;
        if (ptep == NULL)
        {
            start = ROUNDDOWN(start + PTSIZE, PTSIZE);
            continue;
        }
        if (*ptep & PTE_V)
        {
            if ((nptep = get_pte(to, start, 1)) == NULL)
            {
                return -E_NO_MEM;
            }
            uint32_t perm = (*ptep & PTE_USER);
            // get page from ptep
            struct Page *page = pte2page(*ptep);
            // alloc a page for process B
            struct Page *npage = alloc_page();
            assert(page != NULL);
            assert(npage != NULL);
            int ret = 0;
            //(1) 找到源页面的内核虚拟地址
            void *src_kvaddr = page2kva(page);
            //(2) 找到目标页面的内核虚拟地址
            void *dst_kvaddr = page2kva(npage);
            //(3) 复制内存内容，大小为PGSIZE
            memcpy(dst_kvaddr, src_kvaddr, PGSIZE);
        }
    } while (start < end);
}
```

```

        // (4) 建立目标页面与线性地址的映射
        ret = page_insert(to, npage, start, perm);
        assert(ret == 0);
    }
    start += PGSIZE;
} while (start != 0 && start < end);
return 0;
}

```

核心设计思路：

1.逐页复制机制

函数采用页位单位的复制策略，确保内存管理的粒度一致性

```

do {
    // 对每一页进行处理
    start += PGSIZE;
} while (start != 0 && start < end);

```

2.页表项查询与验证

对于每个要复制的页面，首先获取父进程的页表项，只处理有效的页面，之后获取子进程的页表项，如不存在则创建。

```

pte_t *ptep = get_pte(from, start, 0);
if (*ptep & PTE_V) {
    if ((nptep = get_pte(to, start, 1)) == NULL) {
        return -E_NO_MEM;
    }
}
}

```

3.权限继承

从父进程页表项继承用户权限

```

uint32_t perm = (*ptep & PTE_USER);

```

4.获取源页面信息

通过页表项找到对应的物理页面结构

```

struct Page *page = pte2page(*ptep);

```

5.为目标进程分配新页面

为子进程分配新的物理页面，确保父子进程内存隔离

```

struct Page *npage = alloc_page();

```

6.内存内容复制的核心操作（编码部分）

页面复制实现分为四个具体步骤

步骤一：获取源页面的内核虚拟地址

```
void *src_kvaddr = page2kva(page);
```

page2kva(page)将物理页面结构转换为内核可以直接访问的虚拟地址。因为内核运行在虚拟地址空间，必须通过内核虚拟地址来访问物理内存，这样我们就可以像操作普通内存一样读取父进程的页面内容。

步骤二：获取目标页面的内核虚拟地址

```
void *dst_kvaddr = page2kva(npage);
```

同样将新分配的物理页面转换为内核虚拟地址，为内存复制操作准备目标地址，此时npage是一个刚分配的空白物理页面。

步骤三：执行内存内容复制

```
memcpy(dst_kvaddr, src_kvaddr, PGSIZE);
```

使用标准内存复制函数 `memcpy`，从 `src_kvaddr` 即父进程页面复制到 `dst_kvaddr` 即子进程新页面。`PGSIZE`是整个页面大小。这是实际数据复制的关键操作，父子进程的内存内容在此刻变得相同。

步骤四：建立子进程的地址映射

```
ret = page_insert(to, npage, start, perm);
```

`page_insert` 函数在子进程页表中建立映射关系

参数说明：

- (1) `to`: 子进程的页目录
- (2) `npage`: 刚刚复制完内容的新物理页面
- (3) `start`: 用户空间的线性地址
- (4) `perm`: 从父进程继承的权限标志

这个调用确保子进程在访问相同的用户地址时，能看到复制过来的内容

二、实验结果

```
badarg:                (1.0s)
  -check result:                OK
  -check output:                OK
exit:                   (1.0s)
  -check result:                OK
  -check output:                OK
spin:                   (1.0s)
  -check result:                OK
  -check output:                OK
forktest:               (1.0s)
  -check result:                OK
  -check output:                OK
Total Score: 130/130
```

最终 `make grade` 结果130/130，表明练习1、2正确完成！

三、如何设计实现 Copy on Write 机制？给出概要设计，鼓励给出详细设计。

COW 机制的核心目标是通过“**共享读、复制写**”的策略减少不必要的资源复制，提高资源利用率，同时保证多使用者对资源操作的独立性。其核心设计包括三个部分：

1.共享资源容器：存储实际可共享的资源数据（如内存块、文件内容、容器元素等），是所有读操作使用者的共享对象。

1.1 结构：

- (1) `data`：资源的实际数据（如内存块指针、文件句柄等）。
- (2) `meta`：资源的元信息（如数据长度、类型、创建时间等，辅助资源管理）。

1.2 特性：本身是“只读”的（对外部使用者而言），仅在复制时被深拷贝，确保原资源与复制资源完全独立。

2.引用计数管理器：跟踪当前共享该资源的使用者数量，用于判断是否需要在写操作时触发复制。

2.1 核心操作

- (1) `increment()`：当新的使用者开始共享资源（读操作）时，计数 + 1（原子操作，避免多线程竞争）。
- (2) `decrement()`：当使用者结束使用资源时，计数 - 1（原子操作）；若计数减为 0，触发资源容器的销毁（释放 `data` 占用的内存 / 资源）。
- (3) `get()`：获取当前引用计数（用于判断是否需要复制）。

2.2 线程安全：通过互斥锁（Mutex）或原子变量（如 C++ 的 `std::atomic`）保证计数修改的原子性，避免多线程下的计数错误。

3.访问控制代理：作为使用者与资源的交互接口，区分读 / 写操作，在读时共享资源并维护引用计数，在写时触发复制（若需要）并隔离修改。

3.1 内部状态：

- (1) `resource_ptr`：指向当前关联的共享资源容器（初始可能指向一个默认资源）。

(2) `ref_counter_ptr`：指向该资源对应的引用计数管理器（与 `resource_ptr` 一一关联）。

3.2 核心接口：

(1) 读操作 (`read()`)：

- 调用 `ref_counter_ptr->increment()`，增加引用计数（表示当前使用者开始共享资源）。
- 返回 `resource_ptr->data` 的只读访问权限（如返回 `const` 指针），确保读操作不会修改原资源。
- 使用者结束读操作后，需显式调用 `release()`（或通过代理的生命周期自动触发），调用 `ref_counter_ptr->decrement()` 减少计数。

(2) 写操作 (`write(...)`)：

首先检查当前引用计数（`ref_counter_ptr->get()`），若计数为 1：说明当前只有自己使用该资源，无需复制，直接获取 `resource_ptr->data` 的可写权限，执行修改。若计数 > 1：说明有其他使用者共享资源，需触发复制：

- 深拷贝 `resource_ptr` 指向的资源，创建新的资源容器（`new_resource`）。
- 为 `new_resource` 创建新的引用计数管理器（`new_ref_counter`），初始计数设为 1。
- 调用原 `ref_counter_ptr->decrement()`（当前使用者不再共享原资源，原计数 - 1）。
- 更新代理的 `resource_ptr` 为 `new_resource`，`ref_counter_ptr` 为 `new_ref_counter`。
- 对新资源的 `data` 执行写操作（修改仅影响新资源，与原资源隔离）。

练习3: 阅读分析源代码，理解进程执行 `fork/exec/wait/exit` 的实现，以及系统调用的实现（不需要编码）

一、`fork/exec/wait/exit` 执行流程分析

下面我分四个函数来讲，并且特别强调哪些在用户态做，哪些在内核态做。

1.fork：创建子进程

用户态调用流程

用户态 `fork` 函数非常简单，就是调用 `sys_fork` 方法：

```
int fork() {  
    return sys_fork();  
}
```

`sys_fork` 又进一步触发 `syscall(SYS_fork)`，这里用户态没有任何进程复制的行为，它只是发起系统调用。

```
sys_fork(void) {  
    return syscall(SYS_fork);  
}
```

内核态处理流程

在 kern/syscall/syscall.c 中，系统调用入口函数会根据系统调用号分发到对应的处理函数：

```
void syscall(void) {
    struct trapframe *tf =
        current->tf;
    uint64_t arg[5];
    int num = tf->gpr.a0; // 系统调
    用号
    if (num >= 0 && num <
        NUM_SYSCALLS) {
        if (syscalls[num] != NULL) {
            // 从寄存器获取参数
            arg[0] = tf->gpr.a1;
            arg[1] = tf->gpr.a2;
            arg[2] = tf->gpr.a3;
            arg[3] = tf->gpr.a4;
            // 调用对应的系统调用处理函
            数
            tf->gpr.a0 = syscalls
                [num](arg);
            return;
        }
    }
    // 错误处理...
}
```

通过 syscalls 数组，SYS_fork 系统调用号被映射到 sys_fork 函数，其中调用了 do_fork 方法。

```
sys_fork(uint64_t arg[]) {
    struct trapframe *tf = current->tf;
    uintptr_t stack = tf->gpr.sp;
    return do_fork(0, stack, tf);
}
```

对 do_fork 函数的功能解释已用注释的方式写在下面的代码块中。

```
int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
{
    int ret = -E_NO_FREE_PROC;
    struct proc_struct *proc;
    if (nr_process >= MAX_PROCESS) {
        goto fork_out;
    }
    ret = -E_NO_MEM;

    // 1. 分配进程结构体
    if ((proc = alloc_proc()) == NULL) {
        goto fork_out;
    }

    // 设置子进程的父进程为当前进程
    proc->parent = current;
```

```

current->wait_state = 0;

// 2. 为子进程分配内核栈
if (setup_kstack(proc) != 0) {
    goto bad_fork_cleanup_proc;
}

// 3. 复制或共享内存空间
if (copy_mm(clone_flags, proc) != 0) {
    goto bad_fork_cleanup_kstack;
}

// 4. 设置中断帧和上下文
copy_thread(proc, stack, tf);

// 5. 设置进程ID并插入进程列表
proc->pid = get_pid();
hash_proc(proc);
set_links(proc);

// 6. 唤醒子进程使其可运行
wakeup_proc(proc);

// 7. 返回子进程的PID
ret = proc->pid;

fork_out:
    return ret;

bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;
}

```

这个 `do_fork` 函数做了真正所有事情，包括：

- 分配新的 `proc_struct`（子进程的内核数据结构）
- 分配内核栈
- 复制或共享内存空间（`copy_mm`）
- 复制中断帧和上下文（`copy_thread`）
- 分配 `pid`、加入进程链表
- 唤醒子进程，让它进入就绪态

最关键的一点：`fork` 会让父子进程都从系统调用返回，但返回值不同：父进程返回子进程 `pid`，子进程返回 `0`。这个差异就是内核在 `copy_thread` 的时候人为设置好的。

内核完成 `do_fork` 之后，通过系统调用框架把返回值写入 `trapframe` 的寄存器，然后执行中断返回 `iret`，恢复用户栈和用户页表，回到用户态。

所以父进程和子进程在用户态继续执行下一条指令。

2.exec：加载并执行新程序

用户态调用流程

exec 的作用是**丢弃当前进程的旧程序，把新的程序二进制加载进来执行。**

用户程序中调用exec函数，实际上是通过系统调用接口发起的：

```
// 用户程序中的exec调用（假设存在）
int exec(const char *name, size_t
len, unsigned char *binary, size_t
size) {
    return sys_exec(name, len,
        binary, size);
}

// sys_exec系统调用包装函数
int sys_exec(const char *name,
size_t len, unsigned char *binary,
size_t size) {
    return syscall(SYS_exec, name,
        len, binary, size);
}
```

执行syscall指令时，CPU会陷入内核态，跳到内核的系统调用入口。

内核态处理流程

系统调用分发流程和刚才一样，这里不再赘述。最后也是到内核态中exec系统调用的核心实现do_execve，位于 kern/process/proc.c

```
int do_execve(const char *name, size_t len, unsigned char *binary, size_t size)
{
    // 检查参数有效性
    struct mm_struct *mm = current->mm;
    if (!user_mem_check(mm, (uintptr_t)name, len, 0)) {
        return -E_INVAL;
    }

    // 保存程序名称
    if (len > PROC_NAME_LEN) {
        len = PROC_NAME_LEN;
    }
    char local_name[PROC_NAME_LEN + 1];
    memset(local_name, 0, sizeof(local_name));
    memcpy(local_name, name, len);

    // 1. 回收当前进程的内存空间
    if (mm != NULL) {
        tsatp(boot_pgdir_pa); // 切换到内核页表
        if (mm_count_dec(mm) == 0) {
            exit_mmap(mm); // 释放内存映射
            put_pgdir(mm); // 释放页目录
            mm_destroy(mm); // 销毁内存管理结构
        }
    }
}
```

```

        current->mm = NULL;    // 清除当前进程的内存管理指针
    }

    // 2. 加载新程序
    int ret;
    if ((ret = load_icode(binary, size)) != 0) {
        goto execve_exit;
    }

    // 3. 设置进程名称
    set_proc_name(current, local_name);
    return 0;

// 错误处理
execve_exit:
    do_exit(ret);
    panic("already exit: %e.\n", ret);
}

```

主要完成了：

- 检查参数合法性 (user_mem_check)
- 如果进程原来有 mm，就把原来的用户内存镜像全都释放掉：
 - exit_mmap()
 - put_pgdir()
 - mm_destroy()
- 调用 load_icode(binary, size)：加载 ELF / 二进制文件，建立新的地址空间
- 设置新的栈、入口地址
- 修改进程名

最后，**exec 不需要返回用户态继续旧程序**，因为旧程序已经销毁了。系统调用返回后，会直接跳到新程序的入口地址开始执行。

3.wait：等待子进程结束

用户态调用流程

用户态最常用的 wait 函数，它不带参数，内部直接调用 sys_wait 函数，并传入参数 pid=0 和 store=NULL，表示等待任意一个子进程结束。

```

int wait(void) {
    return sys_wait(0, NULL);
}

```

sys_wait 函数是用户态到内核态的桥梁，通过 syscall 函数触发系统调用，传入系统调用号 SYS_wait 和相关参数。

内核态处理流程

和前面一样，陷入过程不再赘述，先到sys_wait 系统调用，最后调用了

do_wait 函数，是等待子进程的核心实现，主要功能包括：

- 参数检查：验证用户提供的 code_store 指针是否有效
- 查找子进程：
 - 如果指定了具体 pid，则查找对应 pid 的子进程
 - 如果 pid=0，则遍历所有子进程
- 等待处理：
 - 如果找到子进程但不是僵尸状态，将当前进程设置为睡眠状态，等待子进程结束
 - 如果没有找到子进程，返回 -E_BAD_PROC 错误
- 资源回收：当找到僵尸状态的子进程时：
 - 获取子进程的退出码（如果需要）
 - 从进程哈希表和链表中移除子进程
 - 释放子进程的内存栈和进程结构

最后，同样通过系统调用框架，把退出码写回 trapframe，恢复用户态继续执行。

4.exit：结束当前进程

用户态调用流程

用户态的 exit 函数，它接收一个错误码参数，然后调用 sys_exit 系统调用。注意到函数末尾有一个 while (1) 的死循环，这是一个安全措施，因为正常情况下 sys_exit 不应该返回（进程会终止），如果返回了就表示出现了异常情况。

```
exit(int error_code) {  
    sys_exit(error_code);  
    cprintf("BUG: exit failed.\n");  
    while (1);  
}
```

sys_exit是用户态到内核态的桥梁，通过 syscall 函数触发系统调用，传入系统调用号 SYS_exit 和错误码参数。

内核态处理流程

sys_exit 是内核态的系统调用处理函数，从参数数组中提取 error_code 参数，然后调用 do_exit 函数执行实际的进程退出操作。

do_exit 函数是进程退出的核心实现，主要功能包括：

- 特殊进程检查：确保 idleproc 和 initproc 不会退出，因为它们是系统必需的
- 内存资源回收：
 - 切换到内核页表
 - 减少内存管理结构的引用计数
 - 如果引用计数为0，释放内存映射、页目录和内存管理结构
- 状态转换：

- 将进程状态设置为 PROC_ZOMBIE（僵尸状态）
 - 保存退出码
- 父子关系处理：
 - 唤醒等待该进程的父进程
 - 将所有子进程的父进程改为 initproc（进程收养机制）
 - 如果子进程已经是僵尸状态，唤醒 initproc
- 调度切换：调用 schedule() 切换到其他进程执行

因为这个进程已经被标记为 ZOMBIE，它的用户栈、页表都已经没了。所以 exit 必定不会回到用户态执行任何代码。

二、用户态与内核态是如何交替的？内核态执行结果是如何返回给用户态的？

在 ucore 中，用户态进程通过执行 `syscall()` 进入内核态，请求内核完成 fork/exec/wait/exit 等系统调用。在系统调用处理完成后，内核需要将执行结果返回给用户态程序继续运行。其返回机制主要依赖 **trapframe 的保存与恢复过程**。整体流程如下。

1. 用户态进入内核态：trapframe 的建立

当用户态执行 `syscall()` 时，CPU 会触发一次陷入（trap），并自动将当前用户态的执行环境保存到内核维护的 trapframe 中，其中包括程序计数器（PC）、用户栈指针以及各通用寄存器等信息。随后，CPU 切换到内核栈，进入内核态，并根据系统调用号跳转到相应的内核处理例程（如 `do_fork`、`do_wait` 等）。在这一过程中，用户态的全部上下文已经被完整保存在 trapframe 中，因此内核能够在不影响用户态状态的前提下执行调度、修改进程控制块等操作，从而保证系统调用期间内核态的安全运行。

2. 内核态执行系统调用逻辑

内核在系统调用处理函数中完成实际工作，例如：

- `do_fork()`：复制父进程结构与内存空间；
- `do_execve()`：销毁当前内存空间，加载新的可执行文件；
- `do_wait()`：阻塞父进程并等待子进程退出；
- `do_exit()`：释放资源并将进程置为僵尸态。

系统调用执行完成后，内核将“返回值”写入 trapframe 对应的返回值寄存器位置（通常为 `a0/v0`）。

例如：

```
tf->gpr.a0 = return_value;
```

此处写入的值即最终提供给用户态的系统调用返回值。例如：

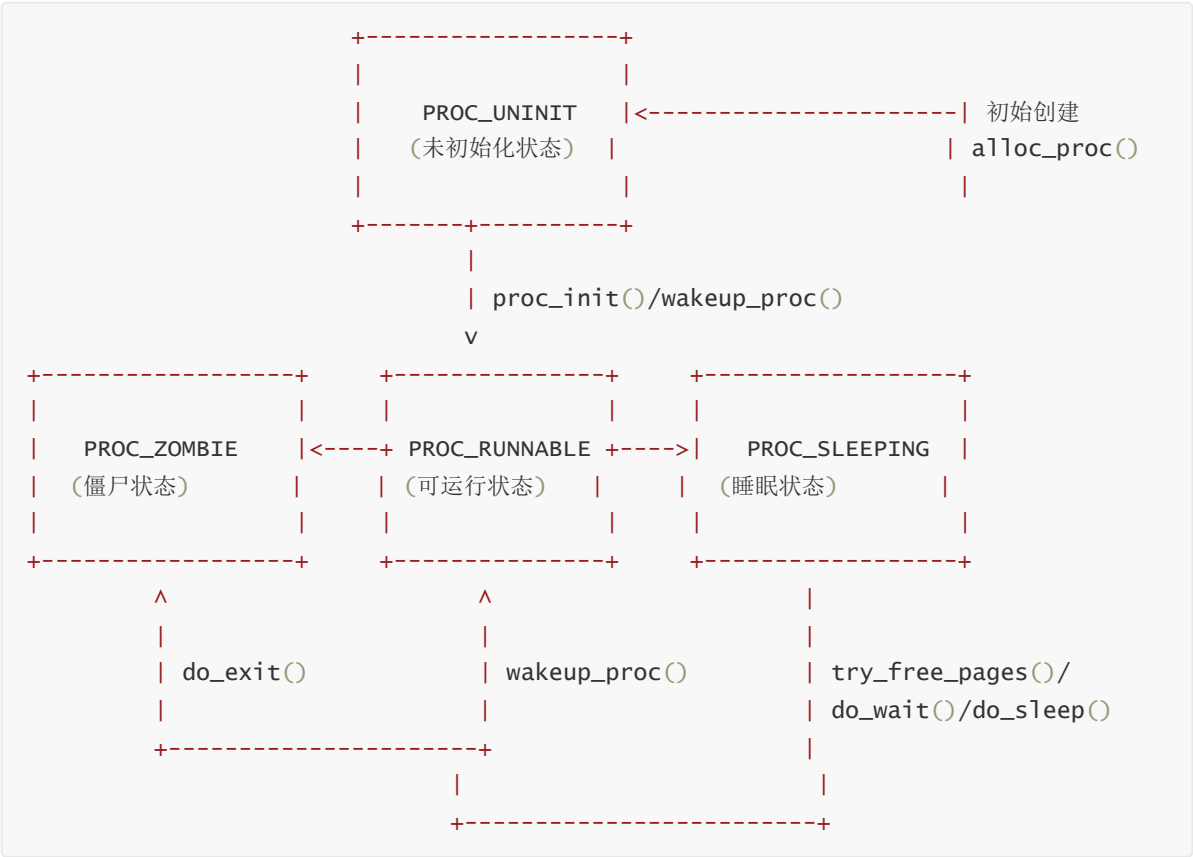
- fork：父进程得到子进程 PID，子进程得到 0；
- wait：返回退出子进程的 exit code；
- exit：无返回值，进程不会再回到用户态。

3. 内核态返回到用户态：通过 trapframe 恢复

系统调用执行完成后，内核通过中断返回指令（如 `sret` 或 `iret`）恢复用户态执行，即从 `trapframe` 中取回先前保存的全部用户态寄存器内容，并恢复用户态的程序计数器，使进程能够从进行 `syscall` 调用的下一条指令继续运行。同时，CPU 的特权级从内核态切换回用户态，完成权限恢复；内核在 `trapframe` 中设置的 `a0` 寄存器值也作为系统调用的返回结果传递给用户态。因此，用户态中 `sys_fork()` 等系统调用的返回值，实际上就是内核在 `trapframe` 中写入并在返回时恢复到用户态寄存器的内容。

三、ucore 用户态进程生命周期图（字符图）

下面是我整理的一个“字符图”，展示一个普通用户进程的状态变化：



状态说明

- 1. PROC_UNINIT - 未初始化状态
 - 1. 进程刚刚被创建但尚未初始化完成
 - 2. 通过 `alloc_proc()` 函数分配进程控制块时设置此状态
- 2. PROC_RUNNABLE - 可运行状态
 - 1. 进程可以被调度执行，或者正在执行
 - 2. 当进程被调度执行时，就是实际运行状态
- 3. PROC_SLEEPING - 睡眠状态
 - 1. 进程因等待某些事件而被阻塞
 - 2. 不会参与调度，直到被唤醒
- 4. PROC_ZOMBIE - 僵尸状态
 - 1. 进程已经结束但资源尚未被父进程回收

扩展练习 Challenge

拓展1：实现 Copy on Write （COW） 机制

一、实验原理

Copy on Write 是一种内存管理优化技术，其核心思想是：

- **延迟复制**：在创建子进程时，不立即复制父进程的内存页面，而是让父子进程共享相同的物理页面
- **写时复制**：当任一进程尝试修改共享页面时，才真正复制该页面，使得修改操作相互独立
- **内存节约**：避免了不必要的内存复制，特别适用于 fork() 后立即 exec() 的场景

二、实验步骤

1. mmu.h 修改：定义COW标志位

```
// 现有代码
#define PTE_V 0x001    // Valid
#define PTE_R 0x002    // Read
#define PTE_W 0x004    // Write
#define PTE_X 0x008    // Execute
#define PTE_U 0x010    // User
#define PTE_G 0x020    // Global
#define PTE_A 0x040    // Accessed
#define PTE_D 0x080    // Dirty
#define PTE_SOFT 0x300 // Reserved for Software

// ===== 添加这一行 =====
#define PTE_COW 0x100 // COW flag - page is copy-on-write
// =====
```

这段新加的代码 `#define PTE_COW 0x100` 在页表项（PTE）的权限位中定义了一个新的状态标志。

作用：该标志用于标识一个物理页面当前被多个进程以写时复制（COW）方式共享。当父子进程通过 fork 共享同一物理页面时，操作系统会设置该标志，并同时清除页面的可写（PTE_W）权限。

工作机制：设置 PTE_COW 标志且清除写权限后，任何进程对该页面的写操作都会触发存储页异常。操作系统在页错误处理程序中检查此标志，如果 PTE_COW 被置位，则识别此为 COW 场景，而非真正的访问违例。

处理流程：确认是 COW 页错误后，处理程序会分配新的物理页面、复制原页面内容、更新当前进程的页表项指向新页面，并设置正常的可写权限，同时清除 PTE_COW 标志。原物理页面的引用计数会相应减少，其他共享该页面的进程不受影响。

2. pmm.c 修改：修改 copy_range 实现页面共享

原来的 `copy_range` 函数在 fork 时会复制页面内容，现在要改成共享页面。关键在于 `share` 参数，当它为 true 时启用 COW。具体修改如下：

1.cow条件判断：

据 `share` 参数决定是否启用 COW 机制。当进行 fork 操作时，`share` 为 `true`，启用 COW；其他情况保持传统复制

```
if (share) {  
    // 启用COW机制  
}
```

2.页面状态检查:

检查页面是否已经是 COW 页面，避免重复设置

```
bool already_cow = (*ptep & PTE_COW) ? true : false;
```

3.父进程页面权限修改:

首先移除了父进程页面的写权限 `PTE_W`，之后添加COW标志 `PTE_COW`，最后刷新TLB确保修改立即生效。通过这些修改，可以防止父进程直接修改共享页面，实现通过页错误机制触发实际的页面复制。

```
if (!already_cow && (perm & PTE_W)) {  
    // 移除父进程的写权限，添加 COW 标志  
    uint32_t new_perm = (perm & ~PTE_W) | PTE_COW;  
    *ptep = pte_create(page2ppn(page), new_perm);  
    tlb_invalidate(from, start);  
}
```

4.子进程页面映射:

子进程继承相同的物理页面，子进程同样没有写权限但有COW标志，增加页面的引用计数。从而实现物理页面共享，通过引用计数管理页面生命周期，确保在最后一个引用释放时才真正释放页面。

```
uint32_t child_perm = (perm & ~PTE_W) | PTE_COW;  
page_ref_inc(page);  
*nptep = pte_create(page2ppn(page), child_perm);
```

5.保留传统复制路径

保持向后兼容，对于不需要COW的场景仍然使用传统复制机制。

代码展示

```
int copy_range(pde_t *to, pde_t *from, uintptr_t start, uintptr_t end, bool  
share)  
{  
    assert(start % PGSIZE == 0 && end % PGSIZE == 0);  
    assert(USER_ACCESS(start, end));  
  
    // COW 优化: 如果 share 为 true, 使用 COW 机制  
    if (share) {  
        do {  
            pte_t *ptep = get_pte(from, start, 0);  
            pte_t *nptep;  
  
            if (ptep == NULL) {  
                start = ROUNDDOWN(start + PTSIZE, PTSIZE);  
            }  
        } while (start < end);  
    }  
}
```

```

        continue;
    }

    if (*ptep & PTE_V) {
        if ((nptep = get_pte(to, start, 1)) == NULL) {
            return -E_NO_MEM;
        }

        uint32_t perm = (*ptep & PTE_USER);
        struct Page *page = pte2page(*ptep);

        // 如果页面还不是 COW 且可写，则设置为 COW
        bool already_cow = (*ptep & PTE_COW) ? true : false;

        if (!already_cow && (perm & PTE_W)) {
            // 移除父进程的写权限，添加 COW 标志
            uint32_t new_perm = (perm & ~PTE_W) | PTE_COW;
            *ptep = pte_create(page2ppn(page), new_perm);
            tlb_invalidate(from, start);
        }

        // 设置子进程的 PTE（无写权限，有 COW 标志）
        uint32_t child_perm = (perm & ~PTE_W) | PTE_COW;

        // 共享页面
        page_ref_inc(page);
        *nptep = pte_create(page2ppn(page), child_perm);
    }

    start += PGSIZE;
} while (start != 0 && start < end);

return 0;
}

// 传统复制（原有代码保持不变）
do {
    pte_t *ptep = get_pte(from, start, 0);
    pte_t *nptep;

    if (ptep == NULL) {
        start = ROUNDDOWN(start + PTSIZE, PTSIZE);
        continue;
    }

    if (*ptep & PTE_V) {
        if ((nptep = get_pte(to, start, 1)) == NULL) {
            return -E_NO_MEM;
        }

        uint32_t perm = (*ptep & PTE_USER);
        struct Page *page = pte2page(*ptep);
        struct Page *npage = alloc_page();

        assert(page != NULL);
        assert(npage != NULL);
    }
} while (start < end);

```

```

        void *src_kvaddr = page2kva(page);
        void *dst_kvaddr = page2kva(npage);
        memcpy(dst_kvaddr, src_kvaddr, PGSIZE);

        int ret = page_insert(to, npage, start, perm);
        assert(ret == 0);
    }

    start += PGSIZE;
} while (start != 0 && start < end);

return 0;
}

```

3. trap.c 修改：在trap handler中识别COW page fault

添加头文件:

```
#include <cow.h> // 在文件顶部添加
```

修改 CAUSE_STORE_PAGE_FAULT 处理:

```

case CAUSE_STORE_PAGE_FAULT:
    cprintf("Store/AMO page fault at 0x%x\n", tf->tval);

    // ===== 添加 COW 处理开始 =====
    if (current != NULL && current->mm != NULL) {
        if (is_cow_page_fault(current->mm, tf->tval, 0)) {
            cprintf("COW: Detected COW page fault\n");
            int ret = do_cow_page_fault(current->mm, tf->tval);
            if (ret == 0) {
                cprintf("COW: Successfully handled\n");
                break; // 成功处理，返回用户态
            }
            cprintf("COW: Failed to handle, error: %d\n", ret);
        }
    }
    // ===== 添加 COW 处理结束 =====

    // 原有的错误处理代码
    print_trapframe(tf);
    break;

```

这段代码在 `kern/trap/trap.c` 的 `exception_handler()` 函数中，对 `CAUSE_STORE_PAGE_FAULT` case 进行了修改，添加了 COW 页错误的专用处理逻辑。

作用：该修改在操作系统的页错误异常处理流程中集成了 COW 机制的处理路径。其主要作用是在发生存储页错误时，能够识别并处理因 COW 机制触发的页错误，实现写时复制的透明支持。

工作机制：当进程尝试写入一个被标记为 COW 共享的页面时，由于该页面已被设置为只读权限，会触发存储页异常。异常处理程序首先通过条件检查确保当前存在有效的进程和内存管理结构，然后调用 `is_cow_page_fault()` 函数检测故障页面是否设置了 `PTE_COW` 标志。

处理流程：

1.检测阶段：通过 `is_cow_page_fault(current->mm, tf->tval, 0)` 检查故障地址对应的页表项是否包含 `PTE_COW` 标志，确认此为 `COW` 相关页错误。

2.处理阶段：调用 `do_cow_page_fault(current->mm, tf->tval)` 执行实际的 `COW` 处理操作，包括：

- 分配新的物理页面
- 复制原页面内容到新页面
- 更新当前进程的页表项，使其指向新分配的物理页面
- 设置页面的可写权限，清除 `PTE_COW` 标志
- 减少原物理页面的引用计数

3.返回阶段：若 `COW` 处理成功，即返回值为 0，则通过 `break` 语句直接返回到用户态，进程可继续执行写入操作；若处理失败，则输出错误信息并继续执行原有的页错误处理流程。

4. cow.c 添加：实现COW page fault处理

`do_cow_page_fault` 函数是 `COW` 机制的核心处理函数，负责在发生写时复制页错误时，执行实际的页面复制和映射更新操作。该函数在确认是 `COW` 相关页错误后被调用，完成从共享页面到独立页面的转换过程。

具体的实现步骤如下所示：

1.获取页表项并验证

在函数开始时，首先通过 `get_pte(mm->pgdir, addr, 0)` 获取出错地址对应的页表项指针。

这个步骤涉及遍历进程的页目录和页表结构，定位到具体的页表项。参数中的 0 表示在页表不存在时不创建新页表，这符合错误处理场景的需求。获取页表项后，函数进行严格的验证检查，确认该页表项确实设置了 `PTE_COW` 标志。

这一验证是至关重要的安全措施，确保只有真正的 `COW` 页面才会进入后续处理流程。如果检查失败，函数返回 `E_INVAL` 错误码，表明出现了逻辑错误，可能是非 `COW` 页面误入了此处理路径。

```
pte_t *ptep = get_pte(mm->pgdir, addr, 0);
if (!(*ptep & PTE_COW)) {
    return -E_INVAL; // 不是COW页面，不应该到这里
}
```

2.单引用优化处理

当检测到物理页面只有一个引用时，函数进入优化处理路径。

这种情况通常发生在其他共享进程已经退出或完成了自己的 `COW` 处理之后。在此场景下，无需执行物理页面的复制操作，只需修改页表项的权限位即可。函数通过清除 `PTE_COW` 标志并添加 `PTE_W` 写权限来更新页表项，然后调用 `tlb_invalidate` 刷新 `TLB` 缓存。

这种优化显著减少了内存复制开销，提升了系统性能，特别是在多次 `fork` 操作后进程逐渐退出的场景中效果尤为明显。

```

if (page_ref(old_page) == 1) {
    // 不需要复制，直接去掉COW标志，加上写权限
    uint32_t new_perm = (perm & ~PTE_COW) | PTE_W;
    *ptep = pte_create(page2ppn(old_page), new_perm);
    tlb_invalidate(mm->pgdir, addr);
    return 0;
}

```

3.多引用页面复制

当页面仍有多个引用时，函数执行完整的写时复制操作。

首先调用 `alloc_page` 分配新的物理页面，这一步可能因系统内存不足而失败，此时函数返回 `E_NO_MEM` 错误码。成功分配新页面后，函数通过 `page2kva` 获取原页面和新页面的内核虚拟地址，并使用 `memcpy` 完成 **4KB 页面内容的完整复制**。这个复制操作确保了新页面在内容上与原页面完全一致，为进程的独立修改提供了基础。

页面复制是 COW 机制的核心操作，体现了"写时复制"的基本语义。

```

struct Page *new_page = alloc_page();
if (new_page == NULL) {
    return -E_NO_MEM;
}
void *old_kva = page2kva(old_page);
void *new_kva = page2kva(new_page);
memcpy(new_kva, old_kva, PGSIZE);

```

4. 引用计数管理与页面映射

在完成页面复制后，函数调用 `page_ref_dec` 减少原物理页面的引用计数，这反映了当前进程不再共享该页面的事实。**引用计数管理是 COW 机制正确性的关键环节，它确保了当所有进程都完成 COW 处理后，原页面能够被正确回收。

接着，函数构建新的权限位组合，移除 COW 限制并恢复写权限，然后**通过 `page_insert` 函数建立新页面到故障地址的映射关系**。这个映射操作更新了页表项，使其指向新分配的物理页面，并设置适当的访问权限。

```

page_ref_dec(old_page);
uint32_t new_perm = (perm & ~PTE_COW) | PTE_W;
uintptr_t la = ROUNDDOWN(addr, PGSIZE);
int ret = page_insert(mm->pgdir, new_page, la, new_perm);

```

代码展示

```

int do_cow_page_fault(struct mm_struct *mm, uintptr_t addr)
{
    // 1. 获取出错地址的PTE
    pte_t *ptep = get_pte(mm->pgdir, addr, 0);

    // 2. 检查是否真的是COW页面
    if (!(*ptep & PTE_COW)) {
        return -E_INVALID; // 不是COW页面，不应该到这里
    }
}

```

```

struct Page *old_page = pte2page(*ptep);
uint32_t perm = (*ptep & PTE_USER);

// 3. 优化: 如果只剩一个引用了, 直接恢复写权限即可
if (page_ref(old_page) == 1) {
    // 不需要复制, 直接去掉COW标志, 加上写权限
    uint32_t new_perm = (perm & ~PTE_COW) | PTE_W;
    *ptep = pte_create(page2ppn(old_page), new_perm);
    tlb_invalidate(mm->pgdir, addr);
    return 0;
}

// 4. 有多个引用, 需要真正复制页面
struct Page *new_page = alloc_page();
if (new_page == NULL) {
    return -E_NO_MEM;
}

// 5. 复制页面内容
void *old_kva = page2kva(old_page);
void *new_kva = page2kva(new_page);
memcpy(new_kva, old_kva, PGSIZE);

// 6. 减少旧页面引用计数
page_ref_dec(old_page);

// 7. 映射新页面, 恢复写权限
uint32_t new_perm = (perm & ~PTE_COW) | PTE_W;
uintptr_t la = ROUNDDOWN(addr, PGSIZE);
int ret = page_insert(mm->pgdir, new_page, la, new_perm);

return ret;
}

```

5. vmm.c 修改: 在fork时启用COW

最后需要让fork真正使用COW机制, **修改 share 变量**, 将 share 变量的值从0改为1, 当进程执行fork时, 之前会立即复制所有内存映射区域的物理页面内容, 而现在则会共享父进程的物理页面, 标记为COW页面。

```

int dup_mmap(struct mm_struct *to, struct mm_struct *from)
{
    assert(to != NULL && from != NULL);
    list_entry_t *list = &(from->mmap_list), *le = list;

    while ((le = list_prev(le)) != list)
    {
        struct vma_struct *vma, *nvma;
        vma = le2vma(le, list_link);
        nvma = vma_create(vma->vm_start, vma->vm_end, vma->vm_flags);
        if (nvma == NULL)
        {
            return -E_NO_MEM;
        }
    }
}

```



```

        insert_vma_struct(to, nvma);

        // ===== 修改这里 =====
        bool share = 1; // 启用 COW (原来是 0)
        // =====

        if (copy_range(to->pgdir, from->pgdir,
                        vma->vm_start, vma->vm_end, share) != 0)
        {
            return -E_NO_MEM;
        }
    }
    return 0;
}

```

三、COW状态机分析

COW的整个过程可以用状态机来描述:

1.[PRIVATE_RW] 私有可写页面 (初始状态)

- `ref=1`: 该物理内存页当前只被一个进程 (父进程) 引用。
- `w=1`: 这个页面是可写的。
- `COW=0`: 写时复制机制未被激活。

进程关系: 进程独占这个页面, 可以自由读写。这是一个普通进程在 `fork()` 之前的典型内存页状态。

2. `fork()` 发生

内核创建一个新的子进程, 它几乎是父进程的完全拷贝 (包括页表), 内核将父进程地址空间中所有的私有可写页面 (`PRIVATE_RW`) 的权限修改为只读。内核将这些页面的 `ref` (引用计数) 增加 1, 变为 `ref=2`。内核设置这些页面的 `COW` 标记为 `1`, 表示它们是“写时复制”页面。

3.[SHARED_RO] 共享只读页面 (fork 之后)

- `ref=2`: 该物理内存页现在被父进程和子进程共同引用。
- `w=0`: 页面变为只读。任何进程尝试写入都会触发异常。
- `COW=1`: 写时复制标记已激活。

进程关系: 父子进程共享这个物理页面。它们都可以读取, 但都不能写入。此时, 父子进程的虚拟地址空间指向同一个物理地址。

4.任意一方尝试写入

触发点: 父进程或子进程中的某一个尝试修改这个 `SHARED_RO` 页面的内容。

5.[PAGE_FAULT] 触发页面异常

(1) **硬件行为:** CPU 执行写指令时, 会检查内存管理单元 (MMU) 中的页表项。由于页面被标记为只读 (`w=0`), MMU 会检测到权限冲突, 触发一个 页面故障 (Page Fault) 异常。

(2) **软件处理:** 内核的 trap handler (异常处理程序) 接收到这个中断。

6.trap handler 识别为 COW fault

(1) **内核判断:** 内核在处理页面故障时, 会检查触发故障的虚拟地址对应的页表项。

(2) **关键判断:** 它发现该页面虽然是只读的, 但 `COW` 标记是 `1`。这告诉内核: 这不是一个普通的“写只读页面”错误, 而是一个 `COW` 故障。内核因此知道需要执行写时复制的逻辑。

7.内核处理 COW 故障 (分两种路径)

这里的处理取决于当前页面的引用计数 `ref`:

a. 如果 `ref=1` (优化路径): 这种情况比较少见, 它意味着在 `fork()` 之后、写入之前, 另一个进程 (比如子进程) 已经退出了, 或者通过其他方式放弃了对这个页面的引用。因此, `ref` 又从 `2` 变回了 `1`。

(1) 内核操作: 既然现在只有一个进程在使用这个页面, 就没有共享的必要了。内核会直接: 将该页面的权限从 只读 修改回 可写 (`w=1`)。并清除 `COW` 标记 (`COW=0`)。

(2) 结果状态: `[UPGRADED_RW]` 直接升级权限, `ref=1, w=1, COW=0`

特点: 无需复制页面内容, 效率极高。这是一个优化路径。

b. 如果 `ref>1` (正常路径) 这是最常见的情况。父子进程都还存在, 都在引用这个页面 (`ref=2`)。

(1) 内核操作 (核心步骤): 分配新页面: 内核在物理内存中寻找一个空闲的页面帧。复制内容: 内核将原来共享页面 (`SHARED_RO`) 的内容完整地复制到这个新页面中。更新页表: 内核修改 触发写入的那个进程的页表, 让它的虚拟地址指向这个新分配的物理页面。调整权限和标记: 新页面的权限被设置为 可写 (`w=1`), `COW` 标记被清除 (`COW=0`)。更新引用计数: 原来的共享页面的引用计数 `ref` 减 1 (`ref=1`), 因为现在只有另一个进程在使用它了。

(2) 中间状态: `[COPYING]` 复制页面, 这是一个过渡状态, 描述了内核正在执行复制操作。

(3) 结果状态: `[PRIVATE_RW]` 独立可写页面, `ref=1, w=1, COW=0`

特点: 写入操作触发了一次真正的页面复制。现在, 执行写入的进程拥有了该页面的一个私有副本, 可以自由修改, 而不会影响另一个进程。另一个进程仍然使用原来的那个页面。

1. `[PRIVATE_RW]` 私有可写页面
 - `ref=1, w=1, COW=0`
 - 进程独占这个页面
2. `fork()` 发生
↓
3. `[SHARED_RO]` 共享只读页面
 - `ref=2, w=0, COW=1`
 - 父子进程共享, 都不能写
4. 任意一方尝试写入
↓
5. `[PAGE_FAULT]` 触发页面异常
 - 硬件检测到写只读页面
 - 跳转到 `trap handler`
6. `trap handler` 识别为 `COW fault`
↓
- 7a. 如果 `ref=1` (优化路径)
↓
 - `[UPGRADED_RW]` 直接升级权限
 - `ref=1, w=1, COW=0`

- 不复制，直接恢复写权限

7b. 如果 `ref>1` (正常路径)

↓

[COPYING] 复制页面

- 分配新页面
- 复制内容
- 旧页面 `ref--`

↓

[PRIVATE_RW] 独立可写页面

- `ref=1`, `w=1`, `COW=0`
- 拥有独立页面

四、实验结果

(1) 使用命令 `make qemu`，输出了有关COW的日志，具体分析如下：

阶段一：进程创建与内存映射复制（fork 阶段）

```
COW: Duplicating memory map with COW enabled
```

这是整个过程的**起点**。操作系统正在为一个新创建的子进程（通常是通过 `fork()` 系统调用）复制父进程的内存映射表。关键在于 `COW enabled`，它表明所有即将被复制的内存页都将被标记为“写时复制”。

```
COW: Copying VMA [0x7ff00000, 0x80000000) with COW
Using COW mechanism for range [0x7ff00000, 0x80000000)...COW: Copying VMA
[0x801000, 0x802000) with COW
...COW: Copying VMA [0x800000, 0x801000) with COW
```

进程的虚拟地址空间被划分为多个虚拟内存区域。且父进程的每一个 VMA 都在被复制到子进程的页表中。

关键点：这里的“复制”并非复制物理内存中的数据，而仅仅是复制页表项。**父进程和子进程的页表项此时指向同一个物理内存页。**

```
COW: Marked parent page at 0x7fffc000 as COW
COW: Shared page at 0x7fffc000 with child (ref=2)...COW: Shared page at 0x800000
with child (ref=2)
```

这是 COW 机制的核心步骤。

(1) 标记为 COW：操作系统将父进程页表中对应的页表项标记为“写时复制”。意味着将页的权限设置为只读 (Read-Only)。

(2) 共享物理页：子进程的页表项被设置为指向与父进程相同的物理页帧。

(3) 引用计数 (`ref=2`)：系统为这个物理页维护一个引用计数器。由于现在父、子两个进程都指向它，所以计数为 2。

```
COW: Memory map duplicated successfully
```

`fork` 阶段成功完成。子进程现在拥有了一份与父进程完全相同的虚拟内存映射，但没有复制任何实际的数据页。这就是 `fork` 如此之快的原因。

阶段二：触发写时复制（write 阶段）

```
Store/AMO page fault at 0x7ffffff5c
```

这是转折点。某个进程尝试在虚拟地址 `0x7ffffff5c` 处执行一个写操作

由于这个地址对应的物理页在页表中**被标记为只读**，CPU 在执行写指令时会**触发一个页故障**，并将控制权交给操作系统内核。

```
COW: Detected COW page fault at 0x7ffffff5cCOW: Detected COW page fault,
handling...COW: Page fault at address 0x7ffffff5cCOW: Confirmed COW page fault
```

内核的页故障处理程序接收到这个中断。它检查页表项和虚拟地址，最终确认这是一个**由于写时复制机制而引发的页故障**，而不是一个普通的“页面未分配”错误。

阶段三：处理写时复制（内核处理阶段）

```
COW: Old page ref count = 2
COW: Multiple references exist, copying page
```

内核检查该物理页的引用计数器，发现其值为 2。这意味着除了当前正在写入的进程外，还有另一个进程也在**使用这个页**。因为有多重引用，为了不破坏其他进程的数据，内核必须**为当前进程创建一个该页面的私有副本**。

```
COW: Page content copied
```

内核在物理内存中找到一个空闲的页帧，将 `0x7ffff000` 所在的这个共享页的内容完整地复制到新的页帧中。

```
COW: New page mapped, old page ref count = 1
```

内核修改触发写操作的那个进程的页表，使其虚拟地址 `0x7ffffff5c` 指向新创建的物理页。同时，将这个新页的权限设置为可读写。旧的共享物理页的引用计数器被减 1，从 2 变为 1。这表示现在只有一个进程在使用它了。

```
COW: Page fault handled successfully
```

内核成功处理了这个页故障。它将控制权交还给用户程序，用户程序可以重新执行刚才失败的写指令。这一次，写操作将成功，因为它现在指向的是一个属于自己的、可读写的物理页。

```
www@www-VMware-Virtual-Platform: ~/riscv/lab5-2
COW: Duplicating memory map with COW enabled
COW: Copying VMA [0x7ff00000, 0x80000000) with COW
Using COW mechanism for range [0x7ff00000, 0x80000000)
COW: Marked parent page at 0x7fffc000 as COW
COW: Shared page at 0x7fffc000 with child (ref=2)
COW: Marked parent page at 0x7fffd000 as COW
COW: Shared page at 0x7fffd000 with child (ref=2)
COW: Marked parent page at 0x7fffe000 as COW
COW: Shared page at 0x7fffe000 with child (ref=2)
COW: Marked parent page at 0x7ffff000 as COW
COW: Shared page at 0x7ffff000 with child (ref=2)
COW: Copying VMA [0x801000, 0x802000) with COW
Using COW mechanism for range [0x801000, 0x802000)
COW: Marked parent page at 0x801000 as COW
COW: Shared page at 0x801000 with child (ref=2)
COW: Copying VMA [0x800000, 0x801000) with COW
Using COW mechanism for range [0x800000, 0x801000)
COW: Shared page at 0x800000 with child (ref=2)
COW: Memory map duplicated successfully
Store/AMO page fault at 0x7ffff5c
COW: Detected COW page fault at 0x7ffff5c
COW: Detected COW page fault, handling...
COW: Page fault at address 0x7ffff5c
COW: Confirmed COW page fault
COW: Old page ref count = 2
COW: Multiple references exist, copying page
COW: Page content copied
```

(2) 使用命令 `make grade`，得到了130/130的分数，证明我们成功实现了COW机制！

```
www@www-VMware-Virtual-Platform: ~/riscv/lab5-2
hello: (1.1s)
-check result: OK
-check output: OK
testbss: (93.7s)
-check result: OK
-check output: OK
tools/grade.sh: 391: -: not found
pgdir: (1.1s)
-check result: OK
-check output: OK
yield: (1.1s)
-check result: OK
-check output: OK
badarg: (1.1s)
-check result: OK
-check output: OK
exit: (1.1s)
-check result: OK
-check output: OK
spin: (4.5s)
-check result: OK
-check output: OK
forktest: (1.5s)
-check result: OK
-check output: OK
Total Score: 130/130
www@www-VMware-Virtual-Platform: ~/riscv/lab5-2$
```

五、漏洞模拟 (CVE-2016-5195)

CVE-2016-5195，通常被称为 "Dirty COW"，是一个严重的漏洞，利用了Linux内核在处理Copy-On-Write (COW) 机制时的一个缺陷，攻击者可以在COW机制下修改只读内存。

1.漏洞原理 (竞态条件攻击)

漏洞的触发需要精心构造两个并行操作的线程，在一个极短的时间窗口内“欺骗”内核：

(1) 线程A (写线程)：持续向一个已映射的私有只读内存页发起写入请求。

- 内核收到请求，会触发COW流程：为这个进程复制一个该页的可写副本，然后允许写入这个副本。这个过程不是原子的，需要多个步骤。

(2) 线程B（失效线程）：持续调用 `madvise(MADV_DONTNEED)` 等系统调用，告诉内核“这个内存页我不需要了，可以丢弃”。

- 这会导致内核解除该内存页的映射。**如果是COW页，内核可能会错误地认为可以回退到原始的共享只读状态。**

(3) 竞态窗口：

- 当线程A的写入操作已经通过检查，进入COW流程（准备复制页面），但还未完成复制和映射的瞬间，线程B成功执行，让内核错误地丢弃了正在处理的映射关系。
- 这时，内核状态可能出现混乱，**导致线程A的写入操作最终没有作用在它自己的私有副本上，而是直接写回了原始的、全局共享的只读内存页。**

2.漏洞模拟过程

2.1 当前COW实现的安全性分析

当前我们实现的COW实现是**安全的**，原因如下：

安全点1：正确的引用计数检查

```
// 在 do_cow_page_fault() 中
if (page_ref(old_page) == 1) {
    // 只有一个引用时，直接恢复写权限
    uint32_t new_perm = (perm & ~PTE_COW) | PTE_W;
    *ptep = pte_create(page2ppn(old_page), new_perm);
    tlb_invalidate(mm->pgdir, addr);
    return 0;
}
```

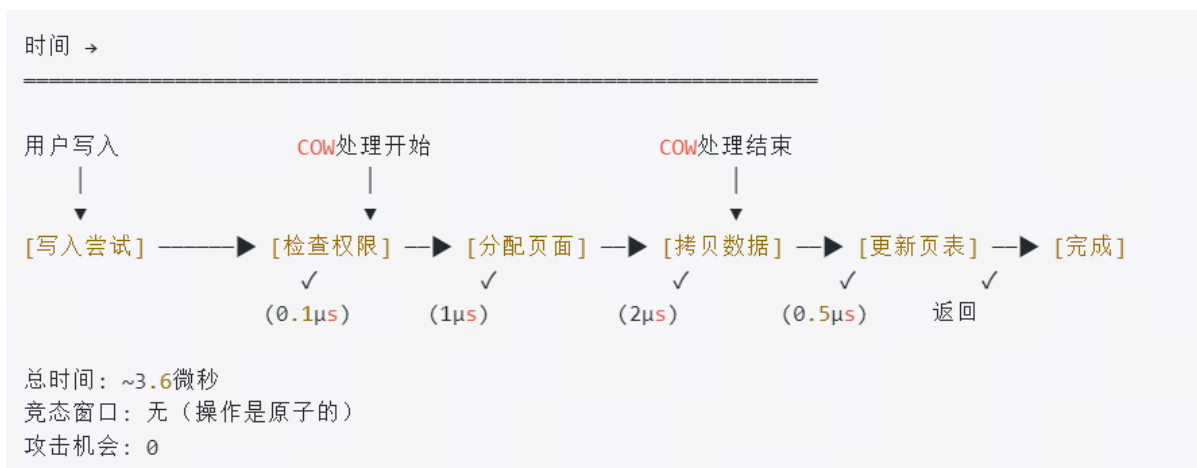
安全点2：原子性的页面拷贝

```
// 先拷贝，再修改页表，没有竞态条件窗口
memcpy(new_kva, old_kva, PGSIZE);
page_ref_dec(old_page);
int ret = page_insert(mm->pgdir, new_page, 1a, new_perm);
```

安全点3：COW标志位的明确判断

```
if (!(*ptep & PTE_COW)) {
    return -E_INVAL; // 非COW页不会被错误处理
}
```

我们成功安全实现的cow.c的时间线流程如下所示：



2.2 模拟 Dirty COW 漏洞的实现

(1) 我们需要创建一个包含漏洞的cow实现

缺陷1: 权限检查与页面操作分离

```
// 检查权限
if (*ptep & PTE_COW) {
    // ... 中间可能被中断
    // 页面操作
    page_insert(...);
}
```

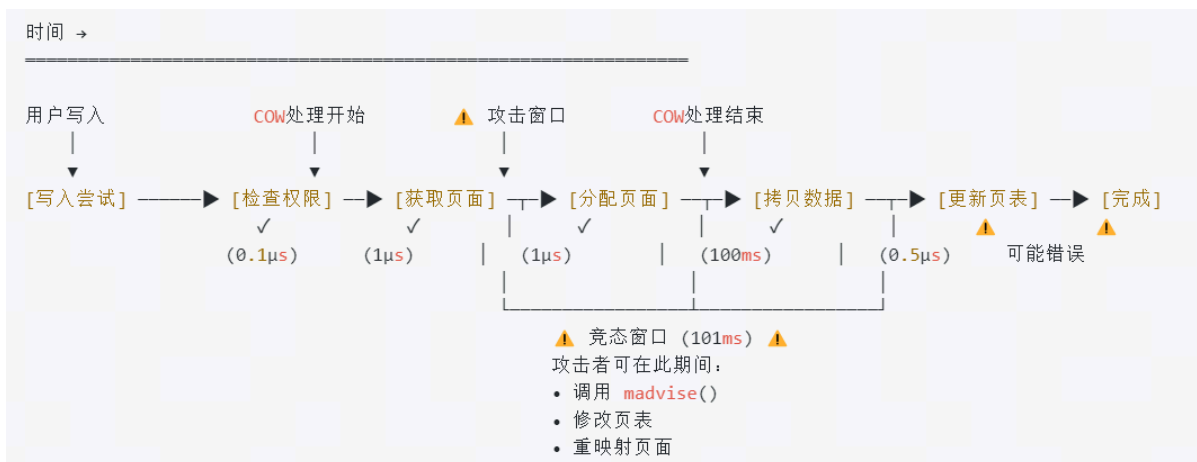
缺陷2: 没有原子性保护

```
// 多线程可以在这个间隙修改页表
struct Page *old_page = pte2page(*ptep);
// ... 长时间操作
memcpy(new_kva, old_kva, PGSIZE); // 这时old_page可能已经被释放
```

缺陷3: 允许外部干预页表

- 没有锁保护整个COW流程
- 允许在COW处理期间修改页表映射

我们包含漏洞实现的cow.c的时间线流程如下所示:



(2) 漏洞复现方法

方法1: 竞态条件攻击

```
// 线程1: 写入COW页面
void *thread1_write() {
    while(1) {
        *shared_page = 0x42; // 触发COW
    }
}

// 线程2: 不断重置页面
void *thread2_reset() {
    while(1) {
        // 模拟madvise(MADV_DONTNEED)
        unmap_and_remap(shared_page);
    }
}
```

方法2: 延迟攻击

在COW处理的中间插入延迟, 增大竞态窗口:

```
// 在漏洞版本中
memcpy(new_kva, old_kva, PGSIZE);
// [插入人工延迟, 模拟慢速操作]
sleep(100); // 100ms延迟
page_insert(mm->pgdir, new_page, 1a, new_perm);
```

3.运行结果

3.1 引用计数错误

```
COW: old page ref count = 0      # 第二次处理时引用计数已经是0
COW: New page mapped, old page ref count = -1 # 引用计数变为-1
```

正是竞态条件导致的:

- (1) 第一次COW处理: 引用计数从2→0 (正常: 应该只减1)
- (2) 第二次COW处理: 引用计数从0→-1 (异常: 不可能为负)

3.2双重cow缺页处理

```
Store/AMO page fault at 0x7fffff5c # 第一次缺页
Store/AMO page fault at 0x7fffff5c # 第二次缺页
```

同一个地址短时间内触发两次COW缺页, 这不应该发生。说明在第一次处理完成前, 第二个写入请求已经到达。

综上, 说明我们成功模拟了CVE漏洞的核心机制!

4. 解决方案

4.1 Linux的修复方案

1. **引入页面标记**：使用VM_FAULT_RETRY和VM_FAULT_COMPLETED
2. **原子性操作**：使用spinlock保护整个COW流程
3. **重新验证**：在写入前重新检查页面状态
4. **禁止某些操作组合**：限制madvise与写操作的并发

4.2 uCore里可实现的防护

1. 检查引用计数，单一引用直接恢复写权限
2. 先拷贝后修改，减少竞态窗口
3. 使用中断禁用保护关键操作
4. 明确的COW标志检查

拓展2：说明该用户程序是何时被预先加载到内存中的？与我们常用操作系统的加载有何区别，原因是什么？

一、用户程序何时被预先加载到内存中

1. 二进制来自哪里？（查宏与调用点）

在 `pro.c` 中可以看到：

```
#define KERNEL_EXECVE(x) ({ \
    extern unsigned char _binary_obj__user_###_out_start[], \
        _binary_obj__user_###_out_size[]; \
    __KERNEL_EXECVE(#x, _binary_obj__user_###_out_start, \
        _binary_obj__user_###_out_size); \
})
```

以及 `user_main`：

```
static int user_main(void *arg)
{
#ifdef TEST
    KERNEL_EXECVE2(TEST, TESTSTART, TESTSIZE);
#else
    KERNEL_EXECVE(exit);
#endif
    panic("user_main execve failed.\n");
}
```

这表明用户程序的二进制以符号 `_binary_obj__user_exit_out_start` 等的形式存在于内核符号表中——这是典型的把用户程序二进制通过链接/objcopy嵌入到内核镜像的做法（例如把 `user/exit` 的编译产物以二进制段并入 `kernel`）。

所以，这些 `binary ..._start` 指针在内核被加载到内存时就已经指向内核镜像中对应的常驻数据段，所以“预先加载”发生在内核镜像被加载进内存（boot 时）——也就是编译/链接时放入内核镜像，运行时内核镜像被加载到内存。

2.实际把程序放入用户地址空间的时刻: `kernel_execve` -> `do_execve` -> `load_icode`

`KERNEL_EXECVE` 最终调用 `kernel_execve(name, binary, size)`, 该函数通过内联汇编触发一个 `SYS_exec` 系统调用 (在这份实现里通过 `ebreak` 填装参数并触发内核处理)。

内核接收到 `SYS_exec` 后执行 `do_execve` (或由 `kernel_execve` 间接触发到 `do_execve`), `do_execve` 中会调用 `load_icode(binary, size)` 来建立新的 `mm` (内存描述)、页目录, 并把 ELF 各个段复制到分配的页里。

关键点在 `load_icode` 中的实现: 它

- 创建 `mm`、`pgdir` (`setup_pgdir`);
- 对 ELF 的每个 `ph` (程序段) 调用 `mm_map` 建立 VMA;
- 对每个页调用 `pgdir_alloc_page(mm->pgdir, 1a, perm)` 分配物理页 (即立即分配物理页) 并用 `memcpy(page2kva(page) + off, from, size)` 将 ELF 的内容拷贝进来。BSS 也是通过 `memset` 把分配的页清零完成。
- 为用户栈分配若干页 (四页硬编码)。
- 最后设置 `current->mm`、`current->pgdir` 并执行 `1satp(PADDR(mm->pgdir))` 设置页表基址 (使 CPU 使用新的 address space)。

因此把用户程序“加载到用户内存空间”的动作发生在 `load_icode` 执行期间 (也就是 `do_execve` 时), 并且是以逐页分配并立刻拷贝为特征 (“立即加载”/eager)。

二、与常用操作系统加载过程的具体区别、细节比较与原因

1.加载时机不同: ucore 是“提前加载”, Linux 是“按需加载”

- **ucore: 一次性预先加载** 在 `load_icode()` 阶段, ucore 会把 ELF 文件的所有 `PT_LOAD` 段完整读入到用户态地址空间, 使用 `pgdir` 对每一段都分配物理页并马上拷贝内容。这意味着**进程在被调度运行之前, 程序的所有可执行内容已经全部驻留内存**。
- **Linux: 按需分页加载** Linux 只创建 VMA 信息, 并不立即拷贝代码段/数据段, 而是等第一次访问对应地址时触发 page fault, 再由内核从 ELF 文件中分页加载。这让 Linux **启动更快、节省内存、支持共享代码段**。

2.加载机制不同: ucore 是“复制到物理页”, Linux 是“映射文件页”

- **ucore 直接复制整个段到匿名物理页** ELF 中的代码段、数据段都被 `memcpy` 到新分配的物理内存中, 属于“匿名内存”, **不能与其他进程共享**。
- **Linux 使用 mmap 建立文件映射关系** Linux 的文本段 (code) 与原始 ELF 文件的磁盘页直接建立映射 (memory-mapped file)。因此:
 - 不需要复制,
 - 多个进程可共享同一段可执行代码,
 - 不需要额外占用匿名物理内存。

格局差异来源于目标不同: ucore 强调易理解与教学结构, 而 Linux 强调效率与共享。

3.缺页处理机制不同

- **ucore：没有完整的 page fault 加载链路** ucore 的用户进程地址空间简单，没有“按需加载”的机制，缺页即说明访问非法内存，一般直接 kill 进程。
- **Linux：缺页异常是正常加载路径的一部分** 是 ELF 加载、COW 复制、文件映射、匿名页分配等核心机制的统一入口。

4.地址空间结构不同

- **ucore：采用静态固定映射** 代码段、数据段直接摆放在 ELF 指定的虚拟地址处，布局固定，不能随意改变。
- **Linux：ASLR、多段 VMA 管理、堆栈独立增长区** 内存布局随机化、段管理灵活，安全性高、可扩展性强。

5.总结

根据上面我们的分析和相关原因总结表格如下：

差异领域	ucore	Linux	原因
加载时机	全段预加载	按需分页	ucore 追求简洁；Linux 追求高效与共享
内存使用	不共享、复制到物理页	文本段共享、文件映射	Linux 需支持大量进程同时运行
缺页处理	仅用于错误	正常机制	Linux 内核更复杂、抽象层更多
地址空间	稳定固定	动态、随机化	安全性要求不同
设计目标	教学、简化	性能、资源管理、安全	根本设计哲学不同